

Domain Rules

The business rules the software enforces: who may do what, what states a request moves through, and how the numbers on the dashboard are defined.

Every rule below cites the file that enforces it, because a rule and its enforcement drift apart the moment either is edited alone.

For how the code is arranged, see [architecture](#); for running it, [local setup](#).

Where this document and the code disagree, the code is right and this document is a bug. Fix it here.

1 At a glance

An associate books a van for a trip. An admin approves or rejects it, assigning a driver and a van. Everyone involved gets email. Every admin action is recorded in an audit trail that cannot be edited from the app.

Two sites operate today, Iloilo and Manila, and the site is a *notification filter*, never a *permission boundary* — see §2.

2 Roles

There are exactly two roles, defined in [src/modules/auth/roles.ts:1](#):

Role	Means
<code>associate</code>	A requestor. Books trips, manages their own.
<code>admin_support</code>	May act on any request, at any site. One flat role.

There is no per-site scoping of admin authority. A Manila admin can approve an Iloilo trip. `ADMIN_SITES (iloilo, manila, all)` tags a whitelist row only to decide who gets cc'd on email — it gates nothing. This is deliberate and stated in three places in the code, because it is the assumption a reader is most likely to get wrong.

How a role is assigned

Role is resolved **at login**, not stored as an authorization fact:

1. The provider authenticates the person against the CGS directory. Only active Carelon employees can authenticate at all — that alone answers "who may sign in".
2. `resolveRole` matches the identity against `admin_whitelist` — Domain ID first, case-folded email as fallback — producing a *capability*: `admin_support` if an active row matches, otherwise `associate`. (`roles.ts:49-73`)
3. The session role is **not** simply that capability. Signing in to the admin portal *requires* `admin_support` and 403s otherwise, before any user row is touched. Signing in to the requestor portal always yields `associate` — even for a whitelisted admin. (`service.ts:81-91`)

So one person can hold both roles, chosen by which portal they sign in to.

`users.role` stores the wider capability, but **nothing authorizes off that column**. Every check reads the session's role.

Revocation is not immediate. The role rides in the session JWT for up to 8 hours. Removing someone from the whitelist takes effect at their next login or when the session expires, whichever comes first. See [operations](#) for what that means for offboarding.

3 The admin whitelist

`admin_whitelist` is the only allowlist left in the system. Because the CGS directory authenticates every active employee, this table is what separates "can sign in" from "can act as an admin".

It controls two things:

- Whether an authenticated employee gets the `admin_support` capability (§2).
- Which admins are cc'd on email for a given trip's site (§9).

A row carries: `fullName`, `email`, `domainId`, `site`, `notify`, `active`, `superAdmin`.

Getting on it

Either seeded from `ADMIN_WHITELIST_SEED` (`admin-whitelist-seed.ts:47`), or created through `/roster` by an existing `super_admin` holder.

The seed re-asserts name, email, domain ID, site, and `super_admin` on every run for the rows it owns — drift correction. It deliberately **never** touches `active` for a row a human has created, deactivated, or reactivated through `/roster`, which it determines from that row's audit provenance. Without that carve-out, every deploy would silently un-retire everyone an admin had retired.

`super_admin`

A boolean, orthogonal to role. Every whitelist row already has `admin_support`; `super_admin` decides who may edit **the whitelist itself** — create, edit, deactivate, reactivate, and grant or revoke the flag on others.

Today two people hold it, by seed: **Ruwi Joy Eribal** (`AG80389`) and **Arielle Jimera** (`AM65108`).

The two-person restriction is a data fact, not a code rule. There is no name check anywhere. Any `super_admin` holder can grant the flag to a third person — an accepted trade-off, noted at `admins/repo.ts:34`. If the restriction needs to be structural, that is a code change, not a documentation one.

One seeded row, **Aishki Hyamero** (`AJ40001`), is a test account and not a member of staff. Leave it alone.

Capability is re-read from the database, every request

`gate()` (`src/app/api/admins/gate.ts:27-49`) reads `super_admin` from the database for the acting session's identity on every `/api/admins` request. It never trusts the `superAdmin` flag in the signed session cookie.

The cookie flag lags in **both** directions: it outlives a revocation, and it misses a grant. The `/roster` page had exactly this bug during development — it gated the whitelist *fetch* on the cookie — which is why the flag is now documented as cosmetic and has no authorization readers.

The four guardrails

All four live inside the repo layer, inside the database transaction, "so no future caller can route around them" (`admins/repo.ts:31-35`).

1. **No self-demotion.** You cannot clear your own `super_admin`. Prevents locking yourself out by accident.

2. **No self-deactivation.** You cannot deactivate your own row — the same lockout by the other door.
3. **Last-holder lock.** Neither demotion nor deactivation may take the count of active `super_admin` holders to zero.
4. **Identity required.** A patch that would leave a row with neither an email nor a Domain ID is rejected with a field-level 400, mirroring the `admin_whitelist_identity_check` constraint before Postgres reaches it.

Guardrail 3 is the subtle one, and worth understanding before touching that code. It takes `pg_advisory_xact_lock(8410372)` before any row lock. A row lock alone is insufficient: two transactions demoting *different* holders never contend on a row, so under READ COMMITTED both would read "2 holders left" from their own snapshot and both would proceed — leaving zero. The advisory lock serializes every write that could shrink the count, which is what makes the `count ≤ 1` check answer truthfully at commit time.

The lock key `8410372` is a global resource guarded only by a comment. A second, unrelated advisory lock reusing that integer would serialize against whitelist writes for no reason.

`isOwnRow` matches on **either** Domain ID or case-folded email — broader than login's matcher, which picks one winner. Deliberately over-inclusive, so a mistyped Domain ID on your own row still counts as yourself for guardrails 1 and 2.

The roster spec lists only three guardrails. Guardrail 4 exists in code and is absent from that spec.

4 Reservation lifecycle

Five statuses, in `src/modules/reservations/types.ts:71-78`:

Status	Database value	Meaning
Pending	<code>pending</code>	Submitted, awaiting a decision
Approved	<code>approved</code>	Approved, driver and van assigned
Approved - Driver Reassigned	<code>approved_reassigned</code>	Approved; an assignment moved afterwards
Rejected	<code>rejected</code>	Declined, with a reason
Cancelled	<code>cancelled</code>	Withdrawn

Approved - Driver Reassigned is **admin bookkeeping only**. `requestorFacingStatus()` collapses it back to Approved on every requestor-facing surface — a requestor never sees the word "Reassigned". They learn of the change from a `driver-assignment` email instead.

Two derived sets matter when reading the code:

- **SCHEDULED_STATUSES** = Pending, Approved, Approved - Driver Reassigned. Statuses that hold a van and driver's time. Drives the calendar, driver workload, and cancellability.
- **APPROVED_STATUSES** = Approved, Approved - Driver Reassigned. "Approved in substance, whichever spelling." Drives reassignability.

Transitions

All enforced server-side in `src/modules/reservations/write.ts`.

From	To	Who	Requires
Pending	Approved	admin	An assigned driver and van (roster or rental)
Pending	Rejected	admin	A non-blank rejection reason
Approved · Reassigned	Approved - Driver Reassigned	admin	Driver or van identity actually moved
Pending · Approved · Reassigned	Cancelled	admin (any) or requestor (own only)	—

Rejected and **Cancelled** are terminal. No path leads out of them.

The approve requirements are mirrored by database CHECK constraints (`reservations_approved_driver_check`, `reservations_approved_van_check`), so the rule holds even against a direct SQL write.

Reassignment is status-sensitive to identity, not to edits. The status flips only if the driver or van actually *moved*. Re-saving the same assignment, or correcting a rental driver's mobile number, leaves the status alone.

A requestor cancelling someone else's request gets a 404, not a 403 — deliberately indistinguishable from a genuinely missing record, so an associate cannot enumerate reference numbers.

All writes use optimistic concurrency. A `version` column compare-and-set; a zero-row update surfaces as `VERSION_CONFLICT` rather than a silent overwrite.

A requestor may edit trip *fields* only while the request is Pending.

5 TAT and SLA

TAT is the gap from submission to the **first** driver assignment. The first one counts even if the driver is later reassigned — a reassignment does not restart or reset the clock.

- **Clock start:** the reservation's `submittedAt`.
- **Clock stop:** the earliest `driver_assigned` event, computed as `min(created_at)` over that reservation's events.
- **Computed by:** `tatHours()` in `dashboard-metrics.ts:30-37`.

`tatHours` returns `null` — excluding the trip from the TAT and SLA population entirely — when the trip was never assigned, either timestamp is unparseable, or assignment somehow precedes submission.

SLA threshold is 12 hours. A trip is within SLA when its TAT is 12 hours or less. `slaSummary()` aggregates to `{assigned, withinSla, avgTatHours, slaPct}`.

TAT and SLA are product decisions, not requirements. Neither term appears in the SRS. The definitions above were chosen by this project, and the original spec recorded them as deferred pending stakeholder input that never arrived. If the client defines TAT differently, this is the code to change.

Van assignment events ([van_assigned](#), [van_reassigned](#)) are recorded in parallel but **no TAT is derived from them**. Only [driver_assigned](#) counts.

6 Trip purposes

18 values, in business order — not alphabetical, and "Others" is deliberately last. Defined in [reference.ts:26-45](#).

External Affairs

Finance Official Travel

HR/TA Related

Office Equipment / Supplies Transfer

IT-Related

Office Transfers

Official Company Event

Safety and Security-Related

ER / Sent Home

SLT Appointments

Team Building

Training-Related

Travel-Related (Airport Transfers)

Facilities/Admin-Related

Team Lunch / Dinner / Event

Asset Retrieval

Onshore/Client Visit

Others

Supplied by the client on 2026-08-27, replacing an earlier six-option list. Three typos in the client's list were corrected in code — "Copany" → Company, "Saftey" → Safety, "Tracel" → Travel. **Do not restore the originals.**

Purpose is not a database CHECK constraint, and that is intentional: nothing branches on the value and the client revises the list. It is stored as plain text, validated only at the write path by [isTripPurpose\(\)](#). A pre-migration row can still hold an off-list value, and the read path tolerates it.

The retired-purposes remap

Migration [20260827225113_remap_retired_purposes.ts](#) maps five values that exist in real production data onto their replacements:

Retired value	Becomes
Airport Transfer	Travel-Related (Airport Transfers)
Finance - Official Travel	Finance Official Travel
HR-Related	HR/TA Related
IT-related	IT-Related
Official Business-related	Others

It is a no-op on a freshly seeded database and **required** before shipping against real data, where all five are in use. Its `down()` is an intentional no-op: "Official Business-related" collapses into "Others" irreversibly, since genuine "Others" rows are afterwards indistinguishable from remapped ones.

7 Vans, drivers and rentals

The split

Vans and drivers were one table. Migration `20260827164450_split_van_from_drivers.ts` dropped `van_number` and `plate` from `drivers` and moved them to a `vans` table, because **a van is assigned per trip, not owned by a driver**. Any driver can take any van on any day.

Van	<code>vanNumber</code> (fleet numbering, <code>VAN-001</code> upward), <code>plate</code> (unique — the real identifier), <code>carType</code> , <code>site</code> , <code>active</code>
Driver	<code>name</code> ("First Last"), <code>mobile</code> , <code>site</code> , <code>shift</code> , <code>active</code>

A reservation points at a driver and a van **independently**, and each side is chosen separately at approval or reassignment time.

`shift` is free text, not a vocabulary. The same migration dropped the old `drivers_shift_check` because the real roster broke it twice over: the four Iloilo drivers have no shift at all, and Manila's are `11AM-11PM` and `11PM-11AM`. Nothing branches on it; it is display-only, and editable without a migration.

Both are deactivated, never deleted. A deactivated driver or van keeps its history and takes no new assignments. Deleting would orphan every trip it ever served.

Rentals

There is **no rentals table**. A rental is per-trip free text on the reservation itself: `rental_driver_name`, `rental_driver_mobile`, `rental_van_number`, `rental_plate`, `rental_car_type`.

Database CHECK constraints enforce the shape:

- Roster and rental are **mutually exclusive per side** — a trip has either an `assigned_driver_id` or a `rental_driver_name`, never both. Same for the van.
- **Paired fields travel together**: `name` $\Leftrightarrow$ `mobile`, `plate` $\Leftrightarrow$ `car type`.
- A van number cannot outlive its plate.

This constrains reporting. A rental driver or van has no roster row, so anything that groups by driver or van identity — driver workload, utilisation — can only count roster-sourced assignments. Rentals appear as text with no stable identity to join or dedupe across trips.

8 Audit trail

The audit log is a **read composition over the write paths' own event tables**, not a table of its own. Every write path already records what it did; a second log would be a second thing to keep in sync ([audit/repo.ts:16-19](#)).

Two sources feed it:

- 1. reservation_events** — every reservation write, appended in the same transaction as the row update. The audit view scopes to `actor_role = 'admin_support'` and to `AUDIT_ACTIONS: approved, rejected, cancelled, modified, driver_assigned, driver_reassigned, van_assigned, van_reassigned`. `submitted` is excluded — this view answers "what did an admin do".
- 2. roster_events** — every roster write (create, update, deactivate, reactivate) to drivers, vans, or the whitelist, via `recordRosterEvent`.

`recordRosterEvent` takes a `Transaction<DB>`, never a plain `Kysely<DB>`, so that **the type system is what prevents logging outside the transaction that made the change**.

Roster audit entries whose target is `admin_whitelist` are visible only to a `super_admin` viewer. A plain `admin_support` reading the audit log does not learn the whitelist's contents through it.

Change diffs

Each entry is `{field, label, from, to}`, where `from` and `to` are **pre-formatted display strings captured at write time**, not raw database values. A row must carry everything needed to read it back years later without re-deriving anything from a schema that has since moved.

`null` means "held nothing", which is distinct from `""`.

Legacy rows may carry only field *names* with no values. `parseChanges()` is a **total function**: any malformed or legacy shape resolves to "no changes" rather than throwing, because a log viewer that throws on one bad row shows nothing at all — the worse failure for an audit trail.

The atomicity rule

A row change and its event are written in one transaction, always. For roster events the type signature enforces it. For reservation events it holds by construction: every write in `write.ts` opens a transaction and inserts its events inside the same callback.

An audit trail with gaps is worse than none, because it is trusted.

9 Notifications

Mail is **enqueued, not sent** during a request. The write path appends to an outbox; a dispatcher drains it separately. See [email and notifications](#) for the pipeline and the templates, and [operations](#) for running the dispatcher.

Domain event	Template	To	Cc
Submitted	<code>booking-submitted,</code> <code>admin-new-request</code>	Requestor, and the site's admins	—
Approved / Rejected	<code>booking-status-</code> <code>change</code>	Requestor	Site admins
Driver or van assigned / changed	<code>driver-assignment</code>	Requestor	Site admins
Cancelled	<code>booking-status-</code> <code>change</code>	Whoever did not act — admin cancelled → requestor; requestor cancelled → admins	Site admins, only when an admin cancelled

Who counts as a site's admins

One function decides, `adminRecipients()` in `recipients.ts`:

- Active rows with `notify` set, whose site is the trip's site or `all`.
- **Sorted by full name**, so the Cc header is stable rather than database-order.
- The acting party's own address is always excluded, case-folded. An admin who is also the requestor is not cc'd on a notice about their own action.
- A whitelist row identified by Domain ID with no email is silently dropped rather than producing a blank address — SES rejects an entire message for one malformed recipient.

`EMAIL_ALWAYS_CC` adds addresses to **every** outbound message, requestor mail included. It is applied in `sendEmail`, the single function every message passes through, so a new template cannot forget it. Deduped against both `to` and `cc` case-insensitively, because SES will happily deliver the same address twice.

See also

- [Architecture](#) — how the code is laid out
- [Operations](#) — environment, deploy, runbook
- [Local setup](#) — getting it running locally